

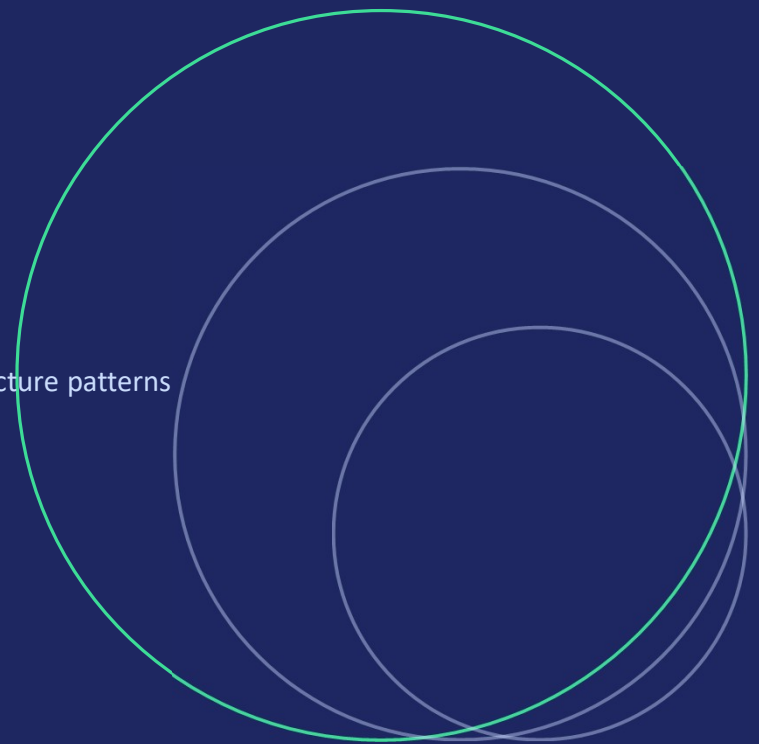
DevOps at Scale

From First Commit to Billions of Transactions

A step-by-step journey through DevOps foundations, engineering practices, and the architecture patterns that power Google-scale systems

Fidelity, Gurgaon | Fresher Onboarding Session

Presented by Abinash Kumar Mishra · Co-Founder & CTO, AceRanker AI



Today's Journey

01

What is DevOps & why it exists

The cultural and engineering problem it solves

02

The Building Blocks

Version control, CI/CD, IaC, containers, orchestration, observability

03

The Scaling Journey

From one server to a global, multi-region platform

04

Google-scale Engineering

SRE, chaos engineering, canary releases at billions/trillions of transactions

05

Getting Started

How you, as freshers, build these skills from day one

What Is DevOps, Really?

Not a tool. Not a job title. A set of practices that closes the gap between building software and running it reliably.



Software Development

- Write code fast
- Ship new features
- Move quickly



IT Operations



- Keep systems stable
- Manage infrastructure
- Minimize risk



DevOps

- Ship fast AND stay stable
- Automate the handoffs
- Shared ownership, feedback loops

Why This Matters More As You Scale

10 users

Manual deploys
are fine

A weekend project

10,000 users

Downtime is noticed; manual deploys are risky

A real product

10 million users

Downtime is expensive; manual anything breaks

A serious business

Billions / trillions of transactions

Every practice in this deck becomes mandatory, not optional

Google / Netflix / Fidelity scale

Part 1

The Building Blocks

The core practices every engineer needs before we talk about scale

Version Control

The foundation: every change to code is tracked, reviewable, and reversible

1

Git as the source of truth

Every engineer works from branches; nothing ships without a commit history

2

Pull requests & code review

Peer review before merge catches bugs early and spreads knowledge

3

Branching strategy (trunk-based / GitFlow)

Keeps releases predictable as teams and codebases grow

Continuous Integration (CI)

Every merge is automatically built and tested — problems surface in minutes, not weeks

1

Automated build on every commit

Catches integration issues immediately, not at release time

2

Automated test suites (unit, integration)

Confidence to change code without manually re-testing everything

3

Fast feedback loop

Jenkins / GitHub Actions / GitLab CI run the pipeline within minutes of a push

Continuous Delivery & Deployment

Getting tested code into production safely, frequently, and with minimal human toil

1

CD pipeline

Packages, tests, and promotes a build through dev → staging → production

2

Deployment strategies

Blue-green, canary, and rolling deployments reduce blast radius of a bad release

3

Feature flags

Ship code dark, then turn features on for a subset of users — decouple deploy from release

Infrastructure as Code (IaC)

Infrastructure is defined in version-controlled code, not clicked together by hand

1

Declarative infrastructure

Terraform / CloudFormation describe the desired end-state of servers, networks, databases

2

Repeatable environments

Spin up an identical staging environment in minutes, not days

3

Auditability

Every infra change goes through the same PR review as application code

Containers & Orchestration

Package an app with everything it needs to run — then let a system manage thousands of them

1

Docker: package once, run anywhere

Eliminates "works on my machine" — the container is identical everywhere

2

Kubernetes: orchestrate at scale

Automatically schedules, restarts, and scales containers across a cluster

3

Service discovery & self-healing

Failed instances are replaced automatically, without a human paging in

Observability & Monitoring

You cannot operate what you cannot see. Metrics, logs, and traces make systems debuggable at scale

1

Metrics & dashboards

Prometheus + Grafana track latency, error rate, throughput (the "golden signals")

2

Centralized logging

ELK / Loki aggregate logs from thousands of instances into one searchable place

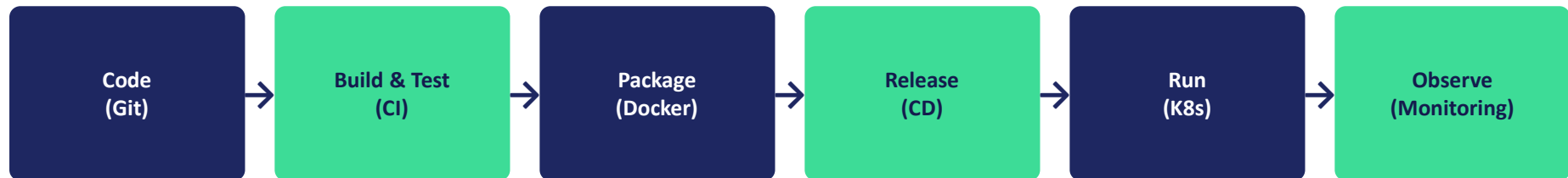
3

Distributed tracing

Follow a single request across dozens of microservices to find where it slowed down

The DevOps Pipeline, End to End

Each building block plugs into the next, forming one continuous flow from code to running system



This entire loop — from a developer's commit to a monitored, running service — should take minutes, not weeks. That speed, done safely, is the whole point of DevOps.

Part 2

The Scaling Journey

How architecture and practices evolve from one server to a global platform

STAGE 1

Scale: hundreds – thousands of users

Single Server / Monolith

One codebase, one deployable unit, one machine. The starting point for almost every product.

- ● App, web server, and database on one box
- ● Simple to build, deploy, and reason about
- ● Manual deploys are still survivable here
- ● Single point of failure — one crash takes everything down



STAGE 2

Scale: tens of thousands of users

Load Balancer + Multiple Servers

Traffic outgrows one box. We add copies of the app behind a load balancer for capacity and redundancy.

- ● Load balancer distributes requests across app servers
- ● Horizontal scaling: add more identical servers, not a bigger one
- ● Database is still often a single instance — the next bottleneck
- ● Health checks auto-remove a failing server from rotation



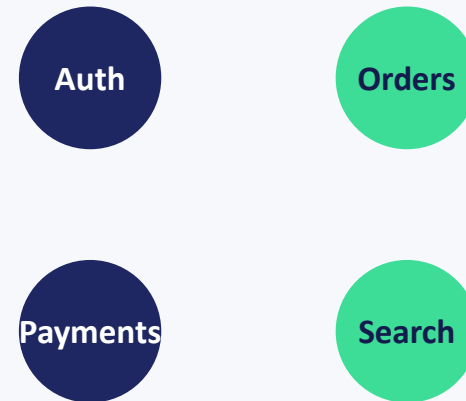
STAGE 3

Scale: millions of users

Microservices Architecture

The monolith is split along business boundaries so teams can build, deploy, and scale services independently.

- Each service owns its own codebase, deploy pipeline, and often its own database
- Teams move independently — no more waiting for one big release train
- API gateway and service mesh manage inter-service traffic and security
- New complexity: distributed systems failures, network latency, versioning



*Independent
services, own DBs*

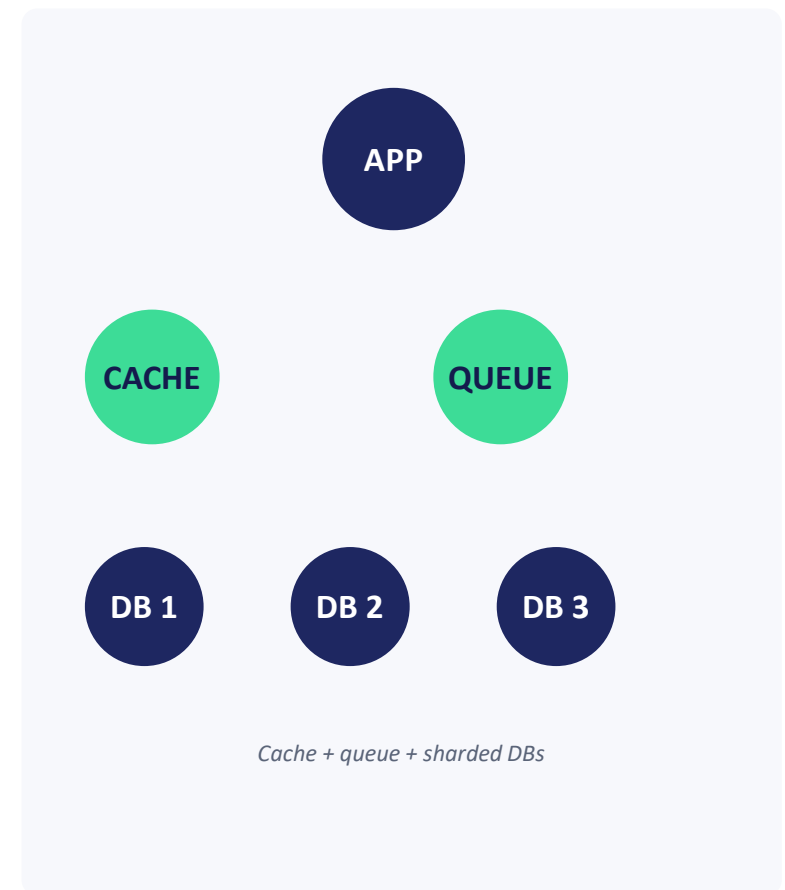
STAGE 4

Scale: tens of millions of users

Data at Scale

The database becomes the bottleneck. We spread reads, writes, and hot data across layers.

- ● Read replicas absorb read-heavy traffic away from the primary
- ● Caching layer (Redis / Memcached) serves hot data in microseconds
- ● Sharding partitions data horizontally across many database nodes
- ● Message queues (Kafka) decouple services and absorb traffic spikes



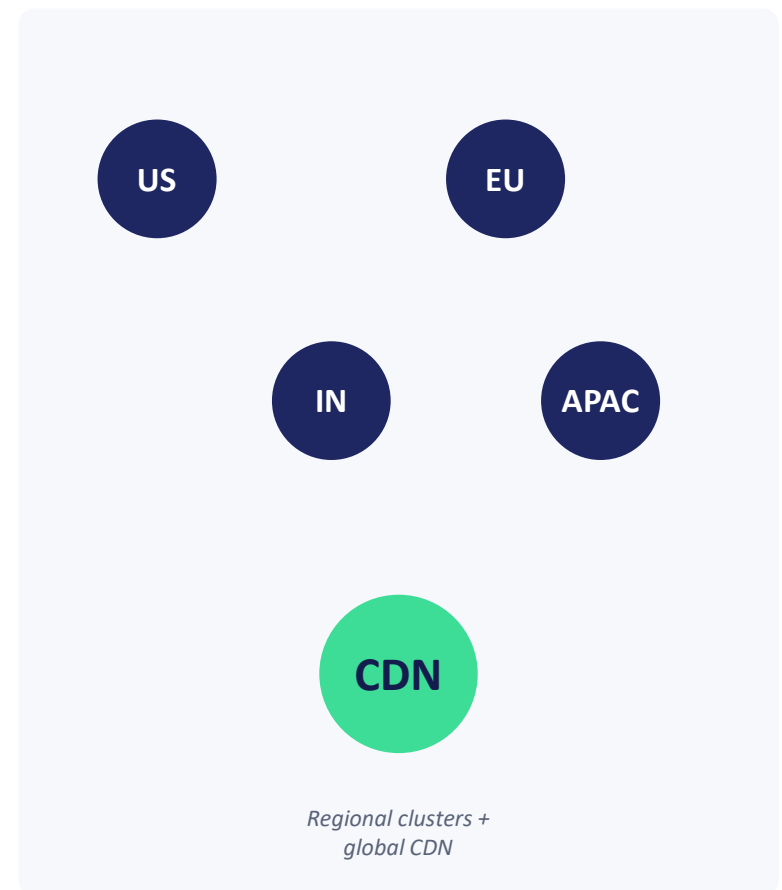
STAGE 5

Scale: hundreds of millions of users

Multi-Region, Global Distribution

One data center isn't enough — for latency, redundancy, and compliance, the platform spans regions.

- CDN edge caching serves static content near the user, worldwide
- Multi-region deployments survive an entire data center outage
- Data replication and consistency models become a first-class design decision
- Global load balancing routes users to the nearest healthy region



Google-Scale Engineering

Billions of users, trillions of requests a day. At this scale, failure isn't an edge case — it's a certainty you engineer around.

Site Reliability Engineering (SRE)

Error budgets balance reliability against feature velocity — teams stop shipping when they burn the budget

Canary releases & progressive rollout

New code hits 1% of traffic, then 5%, then 100% — automatically rolled back on anomaly

Chaos engineering

Deliberately inject failures in production to prove the system survives them before a real outage does

Automated, self-healing infrastructure

Humans design the system; machines detect and remediate failures in seconds

Global traffic & capacity engineering

Predictive autoscaling and multi-region failover absorb traffic spikes without a human paging in

Postmortems, blameless by design

Every incident becomes a permanent system improvement, not a search for someone to blame

Scale, In Numbers

Illustrative order-of-magnitude figures — the point is the jump between tiers, not the exact number

100s

requests/sec

A solid single-server app

100K+

requests/sec

*A national-scale platform (e.g., a large
BFSI system)*

Millions

requests/sec

*A hyperscaler ad-serving or search
pipeline*

Trillions

requests/day

Google-scale global infrastructure

DevOps Maturity: What Changes at Each Level

Practice	Startup	Growth	Enterprise / Google-scale
Deployment frequency	Weekly, manual	Daily, automated	Hundreds/day, continuous
Rollback	Manual redeploy	One-click rollback	Automatic on anomaly detection
Testing	Manual QA pass	Automated CI test suite	Automated + chaos + canary in prod
Monitoring	Server up/down	Dashboards + alerts	SLOs, error budgets, distributed tracing
Incident response	Ad hoc, reactive	On-call rotation	Blameless postmortems, automated remediation

Common Pitfalls to Avoid



Treating DevOps as a tool purchase

It's a practice and a culture change — Jenkins alone doesn't give you CI



Skipping monitoring until something breaks

You can't fix what you can't see; instrument before you need it



Manual, undocumented deployment steps

"Ask Rahul, he knows how to deploy" doesn't scale past one engineer



No rollback plan

Every deployment strategy must answer: how fast can we undo this?



Ignoring the human/process side

Automation without on-call discipline and blameless postmortems still burns teams out

Part 3

Getting Started

Building these skills from your very first sprint at Fidelity

Your First 90 Days: A DevOps Roadmap

Days 1-30

Foundations

- Get fluent in Git branching & PR workflow
- Understand your team's CI pipeline end-to-end
- Read a production runbook / on-call guide

Days 31-60

Contribute

- Ship a small change through the full pipeline yourself
- Write a basic health-check or dashboard for a service
- Shadow an on-call rotation

Days 61-90

Own

- Take a feature from commit to production monitoring
- Participate in a blameless postmortem
- Propose one small automation improvement

Questions?

DevOps is a practice, not a destination — start small, automate one thing at a time, and let reliability compound.

Abinash Kumar Mishra · Co-Founder & CTO, AceRanker AI · @HustlerCoder